

CODEBASE WALKTHROUGH

Market News **AI** Architecture Guide

A file-by-file tour of the .NET daily market-summary pipeline and its admin portal — so nothing stays a black box.

.NET 8 · C#

Playwright scraping

Pluggable AI agents

ASP.NET admin portal

SQLite · EF Core

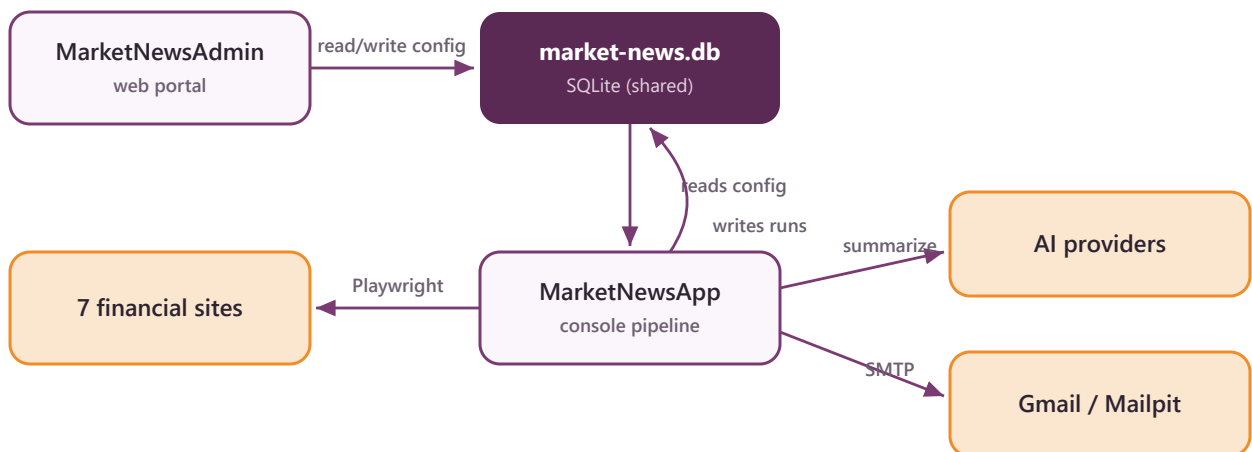
Docker

01 The Big Picture

Two apps share one SQLite database and one library layer.

Component	Role
<code>MarketNewsApp/</code>	Console pipeline — scrape → clean → cache → summarize → email. Runs daily.
<code>MarketNewsAdmin/</code>	ASP.NET MVC web portal — configure the pipeline, view dashboards, browse archived reports.
<code>MarketNewsApp.Tests/</code>	Unit tests.

How data flows



One-sentence mental model: the Admin portal writes config into SQLite, the pipeline reads that config to scrape 7 sites with Playwright, summarizes them via a pluggable AI agent, renders an email, and both apps run as Docker containers sharing one database.

02 Pipeline App — Entry & Services

The console app that does the daily work.

Entry point & config

File	What it does
<code>Program.cs</code>	Main entry. Parses CLI flags (<code>--now</code> , <code>--test</code> , <code>--source</code> , <code>--fresh</code> , <code>--debug-dom</code>), sets up Serilog, runs DB migrations, then drives the 5-step pipeline (<code>RunPipeline</code>). Holds the scheduler loop, checkpoint fallback, and failure-alert email.
<code>MarketNewsApp.csproj</code>	Project file — NuGet deps (Playwright, MailKit, EF Core, Serilog, Scriban).
<code>README.md</code>	Human docs: source list, setup, cache behavior.
<code>Models/Models.cs</code>	Core data shapes: <code>SiteConfig</code> , <code>ScrapedSite</code> , <code>SourceStatus</code> enum, summary types.

Services — the pipeline stages

File	What it does
<code>Scraper.cs</code>	Playwright automation. Loads each site, dismisses overlays, extracts text, and captures chart/table screenshots from the live page (ground truth, never AI-rendered).
<code>AiSummarizer.cs</code>	Per-source Greek AI summaries, then a synthesized overview. Handles transient-error retries.
<code>EmailSender.cs</code>	Renders the Scriban HTML template and sends via SMTP (MailKit). Also sends operational alerts.
<code>ScrapeCache.cs</code>	Caches same-day scrape results by content hash.
<code>SummaryCache.cs</code>	Same idea for AI summaries (per-source + final synthesis).
<code>PipelineCheckpointStore.cs</code>	Persists per-source checkpoints so a crashed run resumes instead of restarting.
<code>ConfigurationService.cs</code>	Loads <code>RuntimeConfiguration</code> (sources, schedule, email, flags) from SQLite.
<code>ReportArchive.cs</code>	Saves generated HTML reports for the admin portal.
<code>AuditLogger.cs</code>	Records audit-trail events.
<code>ProductionMaintenance.cs</code>	Startup housekeeping (DB pragmas, cleanup) before the pipeline.
<code>SmtResilience.cs</code>	Retry/backoff for flaky SMTP sends.

03 AI Agents & Data Layer

Swappable AI providers, plus the EF Core database.

Agents — pluggable AI providers

File	What it does
<code>IChatAgent.cs</code>	Interface every provider implements + <code>ChatMessage</code> record.
<code>ChatAgentFactory.cs</code>	Builds the right agent based on <code>AI_PROVIDER</code> (Copilot default).
<code>CopilotChatAgent.cs</code>	GitHub Copilot SDK provider (default).
<code>AzureOpenAiChatAgent.cs</code>	Azure OpenAI provider.
<code>OpenAiChatAgent.cs</code>	OpenAI provider.
<code>GroqChatAgent.cs</code>	Groq provider.
<code>FailoverChatAgent.cs</code>	Wraps two agents — falls back to a secondary if the primary fails.
<code>OpenAiResilience.cs</code>	Retry/backoff for OpenAI-style calls.
<code>AgentSettings.cs</code>	Config record (model, keys, endpoints) for an agent.
<code>IAgentSettingsProvider.cs</code> · <code>EnvAgentSettingsProvider.cs</code> · <code>SqlAgentSettingsProvider.cs</code>	Where agent settings come from — env vars or the SQLite config.

Data — database layer (EF Core)

File	What it does
<code>MarketNewsDbContext.cs</code>	EF Core context — defines all tables (config, checkpoints, runs, reports).
<code>MarketNewsDbContextFactory.cs</code>	Design-time factory so <code>dotnet ef</code> can create migrations.
<code>ConfigurationEntities.cs</code>	Entity classes for stored configuration.
<code>ConfigurationSeed.cs</code>	Seeds default sources/settings into a fresh DB.
<code>Migrations/*.cs</code>	Ordered schema history (source region, checkpoints, Greek source tuning). Each pair = migration + snapshot.
<code>Templates/email_template.html</code>	Scriban HTML template for the daily report; screenshots embedded as base64.

04 Admin Web Portal

The ASP.NET MVC app for humans to run and observe the pipeline.

Entry & controllers

File	What it does
<code>Program.cs</code>	ASP.NET setup: DI, cookie auth (Administrator role), DB migration, <code>/health</code> endpoint, MVC routing (default → Dashboard).
<code>appsettings*.json</code>	Web app config (SQLite path, logging).
<code>Controllers/DashboardController.cs</code>	Landing dashboard (run stats, health).
<code>Controllers/ManagementController.cs</code>	Edit sources, schedule, email, AI settings; trigger runs.
<code>Controllers/AccountController.cs</code>	Login / logout / access-denied.
<code>Controllers/HomeController.cs</code>	Error page and misc routes.

Services & views

File	What it does
<code>AdminConfigurationService.cs</code>	Reads/writes pipeline config in SQLite (admin side of <code>ConfigurationService</code>).
<code>DashboardService.cs</code>	Aggregates run/health data for the dashboard.
<code>PipelineActivityService.cs</code>	Reads pipeline run history / activity log.
<code>PipelineRunnerService.cs</code>	Launches the pipeline on demand (singleton — one run at a time).
<code>ReportArchiveService.cs</code>	Serves archived HTML reports.
<code>Models/*.cs</code>	View models passed from controllers to Razor views.
<code>Views/</code> + <code>wwwroot/</code>	Razor pages (Account, Dashboard, Management, Home, Shared) + static CSS/JS/images.

05 Tests, Deployment & Artifacts

Everything around the two apps.

Tests

File	What it does
<code>PipelineCheckpointStoreTests.cs</code>	Unit tests for checkpoint save/resume logic.

Deployment & infra (repo root)

File	What it does
<code>compose.yaml</code>	Docker Compose — runs pipeline + admin (+ Mailpit) locally.
<code>Dockerfile.pipeline</code>	Builds the console pipeline container image.
<code>Dockerfile.admin</code>	Builds the admin web container image.
<code>deploy/rancher.yaml</code>	Rancher/Kubernetes deployment manifest.
<code>deploy/chart/</code>	Helm chart for K8s deployment.
<code>.dockerignore</code> · <code>.gitignore</code> · <code>.copilotignore</code>	Exclusion rules for Docker, Git, and Copilot context.
<code>.github/</code>	Copilot instructions, skills, and per-language coding rules.

Generated / disposable (git-ignored)

`artifacts/` , `debug_shots/` , `logs/` , `cache/` , `reports/` , `mailpit-data/` , `report.html` , `synthesis_*.html` , `bin/` , `obj/` — none of these are source; they're build output or test artifacts you can safely delete.